Programowanie gier komputerowych 2025-2026

Wprowadzenie do programowania gier

inż. Mateusz Połeć

Semestr zimowy 2026/2027

Spis treści

1. Czym jest gra?
2. Historia gier
3. Gra vs zwykła aplikacja
4. Architektura silnika gry
5. Popularne silniki gier
6. Game Loop
7. Czas i dt
8. Stan gry
9. Input
10. Rendering
11. Animacje
12. Fizyka i kolizje
13. Audio
14. Multithreading
15. AI
16. Multiplayer

Definicja gry

Programowanie gier to tworzenie interaktywnej aplikacji zsynchronizowanej z urządzeniami wejścia/wyjścia, symulującej spójny stan świata w czasie rzeczywistym na podstawie założonych reguł.
Podstawowe założenia architektury oprogramowania gry są mocno zbliżone do urządzeń wbudowanych (embedded). Analogia polega na konfigurację aplikacji tylko raz (metoda `Setup`), oraz działaniu w nieskończonej pętli (metoda `Frame`).

Historia gier

- 1947 – Analogowy prototyp pierwszej gry (symulator pocisku rakietowego), stworzony za pomocą lamp elektronowych przez Thomasa T. Goldsmitha Jr. i Estle Raya Manna.
- 1952 – *OXO* (kółko i krzyżyk) – jedna z pierwszych gier uruchomionych na komputerze, stworzona przez Alexandra S. Douglasa na komputer EDSAC.
- 1958 – *Tennis for Two* – jedna z pierwszych gier wykorzystujących grafikę na ekranie, stworzona przez Williama Higinbothama na oscyloskopie.
- 1962 – *Spacewar!* – jedna z pierwszych gier komputerowych przeznaczonych do bezpośredniej interakcji wielu graczy; uruchamiana na komputerze PDP-1.
- 1972 – *Pong* – jedna z pierwszych gier arkadowych, która stała się ogromnym sukcesem komercyjnym; początek rozwoju rynku automatów arcade.

Historia gier

- 1977 – Atari 2600 – jedna z pierwszych konsol wykorzystujących wymienne kartridże, umożliwiające uruchamianie wielu różnych gier na jednym urządzeniu.
- 1980 – *Pac-Man* – przełomowa gra arcade, która przyczyniła się do popularyzacji gier wśród masowego odbiorcy.
- 1981 – *3D Monster Maze* – jedna z pierwszych gier wykorzystujących perspektywę pierwszoosobową i środowisko 3D.
- 1985 – *Super Mario Bros.* – jedna z najważniejszych gier platformowych, która pomogła spopularyzować konsolę Nintendo Entertainment System.
- 1992 – *Wolfenstein 3D* – przełomowa gra wykorzystująca techniki renderowania pseudo-3D z perspektywy pierwszej osoby.
- 1993 – *Doom* – kolejny przełom w grafice 3D i grach FPS; popularyzacja rozgrywki sieciowej i dystrybucji shareware.
- 1996 – *Quake* – jedna z pierwszych gier wykorzystujących w pełni trójwymiarowy silnik graficzny oraz sprzętowe przyspieszanie grafiki

Gra vs zwykła aplikacja

Cecha	**Aplikacja**	**Gra komputerowa**
Czas działania	Event-driven	Real-time loop
Logika	Reakcja na dane	Ciągła symulacja świata
Wejście	Klawiatura, mysz	Mysz, klaw., pad, VR
Wyjście	UI, dane, raporty	Obraz, dźwięk, fizyka
Cel działania	Zadanie użytkownika	Rozrywka, immersja
Błędy	Niedopuszczalne	Czasem akceptowalne lub celowe
Synchronizacja	Sporadyczna	Krytyczna (16ms/frame)
Złożoność kodu	CRUD, logika UI	AI, fizyka, rendering, animacje

Architektura silnika gry

Silnik gry to zestaw powiązanych systemów, które współgrają w czasie rzeczywistym.

- Gameplay - logika graczy, AI, interakcje (ECS/Actor based).
- Scripting i Refleksja - Blueprint, RTTI, Invoke.
- Fizyka, kolizje, struktury przestrzenne - Symulacja świata, raycast, AABB, broadphase (Quadtree, BVH, Octree).
- Rendering - pipeline 2D/3D, culling, batching.
- Animacje - blendy, szkielet
- Audio - przestrzenne dźwięki, pogłos, efekty
- Filesystem - zarządzanie plikami (ładowanie assetów, pakiety).
- Multithreading - podział zadań, loading, physics, audio, gameplay threads (Job system, task queue).
- Networking - Multiplayer (TCP/UDP, snapshoty, prediction, rollback, replikacja, P2P-Server sided)
- Czas - pętla gry, deltaTime, fixed update.

Dostępne silniki gier

Społeczność oferuje szeroki wachlarz dostępnych silników do gier – od gigantów rozwijanych przez duże korporacje (np. Unreal Engine, Epic Games), po małe, hobbystyczne rozwiązania open-source (np. Godot, dostępne na `https://github.com/collections/game-engines`).

- Unreal Engine - Rozwijany przez Epic Games, szeroko używany w grach AAA (Fortnite, PUBG, ARK, S.T.A.L.K.E.R 2). Core napisany w C++, Gameplay w Blueprint + C++. Często służy też do tworzenia VFX/wirtualnej scenografii w filmach.
- Unity - Rozwijany przez Unity Technologies, znajduje zastosowanie w grach 2D/3D, mobilnych i VR (m.in. Cuphead). Core napisany w C++/C, Gameplay w C. Bardziej user-friendly na start. Fajny do prototypowania.
- Godot - Open source, rozwijany przez społeczność. Core napisany w C++, Gameplay w GDScript (podobny do Pythona), ale można też w C/C++, Visual Script. Dużo lżejszy niż dwa powyżej, modularny.

Dostępne silniki gier

- In-house - Pisane "od zera"na potrzeby danego studia. Tworzone zazwyczaj w C++ / Rust / C.
  1. RE Engine - Resident Evil.
  2. IW Engine - Call of Duty.
  3. id Tech - DOOM.
  4. RAGE - Grand Theft Auto / Red Dead Redemption.
  5. Frostbite - FIFA 17-23, Battlefield, niektóre Need for Speed.
  6. Creation Engine - The Elder Scrolls V: Skyrim, Starfield, Fallout 4.
- Lightweight - Biblioteki do budowy własnych silników (SDL2, SFML, Raylib, Monogame). Stworzone w C / C++ / C / Rust.

Game Loop



- **Input** – odczytanie działań użytkownika
- **Frame** – aktualizacja stanu świata
- **Render** – wyświetlenie aktualnego stanu

Czas i Δt

- **FPS (Frames Per Second)** – liczba klatek wygenerowanych w ciągu sekundy.
- Im więcej klatek na sekundę, tym częściej ekran jest aktualizowany i tym płynniejszy może być ruch.
- **Frame time** – czas potrzebny na wygenerowanie pojedynczej klatki.
- **Fixed frame rate** – próba utrzymania stałej liczby klatek na sekundę, np. 30, 60 lub 120 FPS.
- W praktyce czas wykonania pojedynczej klatki może się zmieniać w zależności od obciążenia procesora i karty graficznej.

Czas i Δt

Ruch obiektu zależy od **upływu czasu**, a nie od liczby wykonanych klatek.

$$position = position + velocity \cdot \Delta t$$

30 FPS	$\rightarrow$ większe Δt
60 FPS	$\rightarrow$ mniejsze Δt
144 FPS	$\rightarrow$ jeszcze mniejsze Δt

Obiekt porusza się z taką samą prędkością niezależnie od liczby FPS.

Stan gry

Stan gry opisuje **wszystko, co w danym momencie ma znaczenie dla działania i wyświetlania gry**.

Przykładowo:

- położenie i stan gracza,
- położenie i stan przeciwników,
- stan obiektów świata,
- wynik i liczba żyć,
- aktualny poziom,
- stan rozgrywki: menu, gra, pauza, koniec gry.

Game Loop zmienia stan gry, a Renderer go przedstawia.

Input – obsługa urządzeń wejścia

System wejścia (*Input System*) odpowiada za odczytywanie i interpretowanie działań użytkownika.

Przykładowe urządzenia:

- **Klawiatura** – klawisze, kombinacje klawiszy.
- **Mysz** – pozycja kursora, przyciski, rolka.
- **Gamepad** – przyciski, gałki analogowe, triggery.

Obsługa przycisków:

- `OnKeyDown` – przycisk został wciśnięty.
- `OnKeyPressed` – przycisk pozostaje wciśnięty.
- `OnKeyReleased` – przycisk został zwolniony.

Input → Game Logic → Game State

Rendering – od piksela do sceny

- **Pixel** – pojedynczy punkt obrazu, opisany wartością koloru.
- **Framebuffer** – obszar pamięci zawierający obraz przygotowany do wyświetlenia na ekranie.
- **Sprite** – dwuwymiarowy obraz reprezentujący obiekt w grze, np. postać, przeciwnika lub element interfejsu.
- **Texture** – obraz wykorzystywany przez renderer jako dane wizualne, np. powierzchnia obiektu 3D.
- **Mesh** – geometryczny opis obiektu 3D, złożony z wierzchołków i elementów geometrycznych (np. trójkątów).

Mesh + Texture → Obiekt na ekranie

Rendering – jak powstaje obraz?

Scene → Camera → Renderer → Framebuffer → Screen

- **Camera** – określa, z jakiego miejsca i w jakim kierunku obserwujemy świat gry.
- **Shader** – program wykonywany przez GPU, określający sposób przetwarzania geometrii i pikseli.
- **Draw call** – polecenie przekazujące GPU informację, co i jak należy narysować.

Renderer zamienia stan świata gry na obraz.

Rendering – wydajność

Współczesna scena może zawierać tysiące obiektów.

- **Culling** – pomijanie obiektów, których nie trzeba renderować, np. znajdujących się poza polem widzenia kamery.
- **Batching** – łączenie operacji renderowania wielu obiektów w większe grupy w celu ograniczenia liczby operacji wykonywanych przez GPU.

Nie renderuj tego, czego nie trzeba.

Renderuj wiele obiektów możliwie efektywnie.

Animacje – 2D i 3D

Animacja 2D

- zmiana kolejnych klatek obrazu,
- sprite sheet,
- prosta i wydajna,
- typowa dla gier 2D.

2D: zmieniamy obraz

Animacja 3D

- zmiana transformacji modelu,
- animacja kości,
- interpolacja pozycji i rotacji,
- możliwość łączenia wielu animacji.

3D: zmieniamy model

Animacja 2D – Sprite-based Animation

Sprite-based animation polega na wyświetlaniu kolejnych obrazów (*frames*) w odpowiednim tempie.

$$\boxed{\text{Frame 1} \rightarrow \text{Frame 2} \rightarrow \text{Frame 3} \rightarrow \text{Frame 4} \rightarrow \ldots}$$

Często wszystkie klatki są umieszczone w jednym **sprite sheet**.

Animacja 3D – system kości

Zamiast przechowywać osobny obraz dla każdej klatki, model może być animowany za pomocą **szkieletu (skeleton)**.

- model składa się z geometrii (*mesh*),
- wewnątrz modelu znajduje się hierarchia kości,
- animacja zmienia położenie i rotację kości,
- wierzchołki modelu są powiązane z kośćmi.

Animacja kości $\rightarrow$ **deformacja modelu** $\rightarrow$ **rendering**

Fizyka i kolizje

System fizyki odpowiada za symulowanie ruchu obiektów oraz wykrywanie i obsługę ich wzajemnych oddziaływań.

1. **Broad Phase** – szybkie wyszukiwanie par obiektów, które potencjalnie mogą się ze sobą zderzyć.
2. **Narrow Phase** – dokładne sprawdzenie, czy pomiędzy wybranymi obiektami rzeczywiście występuje kolizja.
3. **Collision Response** – określenie skutków kolizji, np. odbicia, zatrzymania lub przesunięcia obiektu.

Broad Phase $\rightarrow$ Narrow Phase $\rightarrow$ Response

Fizyka – 2D i 3D

Do wykrywania kolizji wykorzystuje się uproszczone reprezentacje kształtów (*collision shapes*).

2D

- AABB (Axis-Aligned Bounding Box),
- koło,
- kapsuła,
- wielokąt.

3D

- AABB,
- Sphere,
- Box,
- Capsule,
- Mesh.

Prostszy kształt $\rightarrow$ szybsze obliczenia

Fizyka – optymalizacja kolizji

Nie wszystkie obiekty mogą się ze sobą zderzyć.

1000 obiektów $\longrightarrow$ mniejsza liczba potencjalnych par

Przykładowe struktury przestrzenne:

- **Quadtree** – podział przestrzeni 2D.
- **Octree** – podział przestrzeni 3D.
- **Spatial Grid** – podział przestrzeni na regularną siatkę.

Broad Phase eliminuje oczywiście niemożliwe kolizje.

Audio

System audio odpowiada za odtwarzanie, przetwarzanie i miksowanie dźwięku w czasie rzeczywistym.

- **Efekty dźwiękowe** – kroki, strzały, eksplozje, interakcje.
- **Muzyka** – utwory, ambient, zmiana muzyki zależnie od sytuacji.
- **Dźwięk przestrzenny** – położenie źródła dźwięku względem gracza wpływa na sposób jego odtwarzania.
- **Efekty audio** – np. pogłos, filtracja, echo czy zmiana głośności.
- **Miksowanie** – łączenie wielu źródeł dźwięku i kontrolowanie ich wzajemnych poziomów.

$$\text{Sources} \rightarrow \text{Effects} \rightarrow \text{Mixer} \rightarrow \text{Output}$$

Audio – dźwięk przestrzenny

Źródło dźwięku posiada pozycję w świecie gry.

Źródło dźwięku $\longrightarrow$ Gracz

`lewa` `środek` `prawa`

System audio może uwzględniać m.in.:

- odległość od gracza,
- kierunek, z którego dochodzi dźwięk,
- głośność,
- tłumienie dźwięku przez otoczenie.

Pozycja obiektu w świecie wpływa na słyszany dźwięk.

Audio Middleware

W większych projektach logika audio może być tworzona nie bezpośrednio w kodzie gry, lecz za pomocą narzędzi typu **audio middleware**.

Wwise **FMOD**

Pozwalają m.in. na:

- zarządzanie dużą liczbą dźwięków,
- tworzenie miksów i efektów,
- definiowanie zachowania dźwięku zależnego od stanu gry,
- tworzenie przejść między utworami muzycznymi,
- integrację audio z wydarzeniami i systemami gry.

Multithreading

Współczesna gra wykonuje wiele niezależnych zadań w każdej klatce.

Przykładowe zadania:

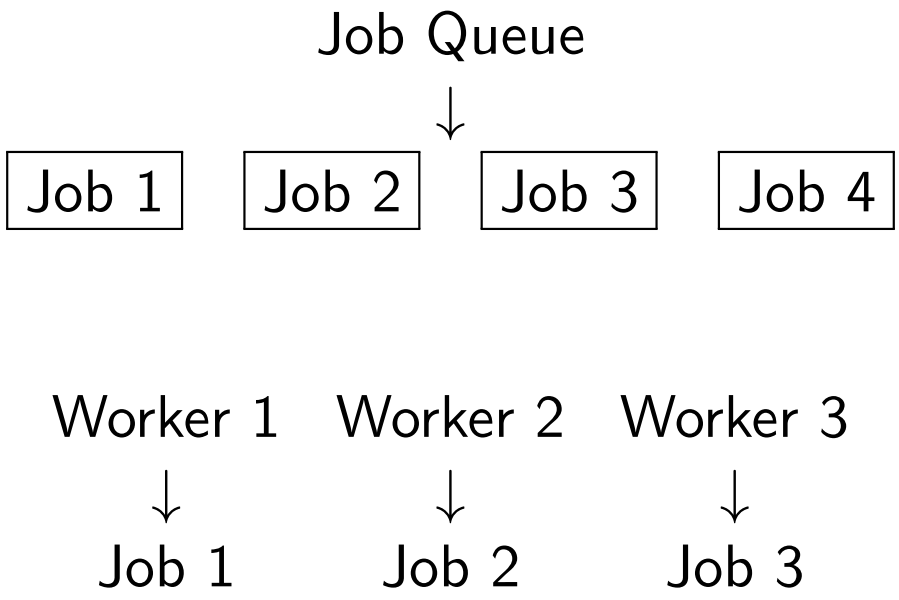
- aktualizacja fizyki,
- animacje,
- sztuczna inteligencja,
- przygotowanie danych do renderowania,
- ładowanie zasobów,
- przetwarzanie dźwięku.

Zamiast wykonywać wszystko sekwencyjnie, zadania mogą być wykonywane równolegle na wielu wątkach CPU.

Game Loop $\rightarrow$ Tasks $\rightarrow$ Worker Threads

Multithreading – Job System

Zamiast przypisywać konkretne zadania do konkretnych wątków, silnik może korzystać z puli wątków roboczych (*worker threads*).



Job System zarządza wykonywaniem małych, niezależnych zadań na dostępnych wątkach procesora.

Multithreading – synchronizacja

Nie wszystkie zadania mogą być wykonane jednocześnie.

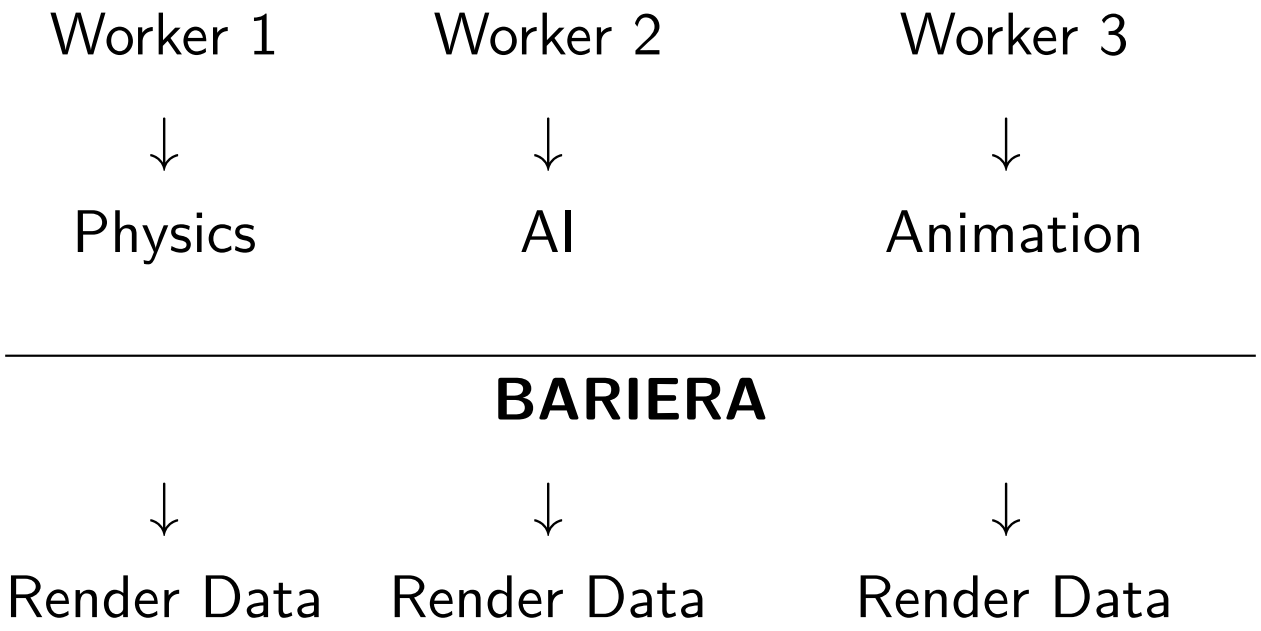
Jeżeli jedno zadanie korzysta z danych utworzonych przez inne zadanie, konieczna jest **synchronizacja**.

Przykładowe mechanizmy:

- mutex / lock,
- atomic operations,
- semafory,
- bariery (*barriers*).

Multithreading – bariera

Bariera synchronizacyjna pozwala wymusić, aby wszystkie zadania zakończyły określony etap przed rozpoczęciem kolejnego.



Następny etap rozpoczyna się dopiero, gdy wszystkie wymagane zadania zostaną zakończone.

Sztuczna inteligencja

AI w grach odpowiada przede wszystkim za podejmowanie decyzji i sterowanie zachowaniem postaci oraz innych obiektów.

Przykładowe decyzje NPC:

- Czy widzę gracza?
- Czy powinienem zaatakować?
- Czy powinienem uciekać?
- Dokąd powinienem się udać?
- Co powinienem zrobić, jeśli gracz zniknie?

Percepcja $\rightarrow$ **Decyzja** $\rightarrow$ **Akcja**

AI – Finite State Machine

Finite State Machine (FSM) opisuje zachowanie jako zbiór stanów oraz przejść pomiędzy nimi.

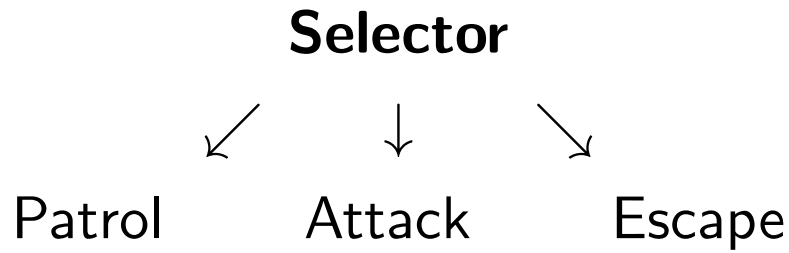




FSM dobrze sprawdza się przy prostych, przewidywalnych zachowaniach.

AI – Behavior Tree

Behavior Tree reprezentuje zachowanie jako hierarchiczne drzewo decyzji i działań.



Przykład:

- Czy jestem zagrożony? → **Escape**
- Czy widzę gracza? → **Attack**
- W przeciwnym przypadku → **Patrol**

Pozwala budować bardziej złożone zachowania z mniejszych elementów.

AI – Blackboard

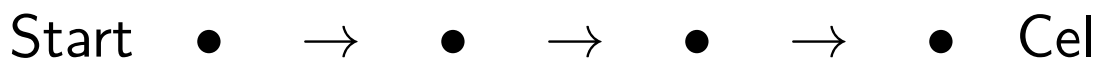
Blackboard przechowuje informacje dostępne dla różnych elementów systemu AI.

`PlayerPosition`	$(120, 80)$
`PlayerVisible`	`true`
`Health`	35
`LastKnownPosition`	$(100, 75)$
`CurrentTarget`	`Player`

Perception $\rightarrow$ $\boxed{\text{Blackboard}}$ $\rightarrow$ Behavior Tree / FSM

AI – Pathfinding

Pathfinding polega na znalezieniu trasy pomiędzy punktem początkowym a docelowym.



Popularne algorytmy:

- **BFS** – przeszukiwanie wszerz,
- **Dijkstra** – najkrótsza ścieżka,
- **A*** – efektywne wyszukiwanie ścieżki z wykorzystaniem heurystyki.

W grach pathfinding często wykonywany jest na specjalnej reprezentacji przestrzeni.

AI – Navmesh

Navigation Mesh (Navmesh) opisuje obszary świata, po których postać może się poruszać.

Zamiast przeszukiwać całą geometrię świata, AI operuje na uproszczonej reprezentacji przestrzeni.

$$\text{Świat gry} \longrightarrow \text{Navmesh} \longrightarrow \text{Pathfinding}$$

- powierzchnie przeznaczone do chodzenia,
- przeszkody i obszary niedostępne,
- połączenia pomiędzy obszarami,
- trasa pomiędzy aktualną pozycją a celem.

Multiplayer

W trybie multiplayer wiele instancji gry musi współdzielić i synchronizować stan świata.

$$\text{Client A} \quad \longleftrightarrow \quad \text{Network} \quad \longleftrightarrow \quad \text{Client B}$$

Problemy:

- opóźnienia sieciowe (*latency*),
- utrata i zmiana kolejności pakietów,
- różne częstotliwości aktualizacji,
- rozbieżności pomiędzy stanami gry.

Każdy gracz może widzieć nieco inną wersję świata.

Multiplayer – architektura

Client–Server



Serwer przechowuje i weryfikuje wspólny stan gry.

Peer-to-Peer



Brak centralnego serwera – uczestnicy komunikują się bezpośrednio między sobą.

Współczesne gry często wykorzystują model Client–Server.

Multiplayer – synchronizacja

Serwer musi regularnie przekazywać klientom informacje o aktualnym stanie świata.



Podstawowe techniki:

- **Snapshot** – okresowy obraz stanu świata.
- **Replication** – synchronizacja wybranych elementów stanu.
- **Interpolation** – płynne przedstawianie otrzymanych danych.
- **Prediction** – przewidywanie przyszłego stanu przez klienta.
- **Rollback** – cofnięcie symulacji i ponowne jej wykonanie po otrzymaniu nowych danych.

Multiplayer – komunikacja

- **TCP** – gwarantuje dostarczenie danych i ich kolejność, ale może wprowadzać dodatkowe opóźnienia.
- **UDP** – nie gwarantuje dostarczenia ani kolejności, ale pozwala aplikacji samodzielnie decydować, jak obsługiwać utracone lub opóźnione dane.

W grach liczy się nie tylko poprawność danych, ale również czas ich otrzymania.

Wydajność i profiling

W grach czas jest zasobem – wszystkie systemy muszą zmieścić się w czasie przeznaczonym na wygenerowanie jednej klatki.

$$60\ \mathrm{FPS} \quad \Longrightarrow \quad \frac{1}{60}\ \mathrm{s} \approx \boxed{16{,}67\ \mathrm{ms}}$$

W tych kilku milisekundach muszą zmieścić się m.in.:

- Input,
- Game Logic i AI,
- Fizyka i kolizje,
- Animacje,
- Audio,
- przygotowanie i wykonanie renderowania.